

FinTechCo Q&A; Battle Card

v4 Supplement | Applied AI Architect Interview Prep

Audience: CTO (skeptical, security-first, 20+ yrs fintech) · Head of Digital Transformation (champion, wants productivity wins)

This guide supplements your DEMO_PREP_v4.md (demo flow), DEMO_CHEATSHEET.md (run-of-show), and INTERVIEW_PREP.md (full prep pack). It focuses on the six feedback areas your practice run surfaced.

FEEDBACK AREA 1

"How does the data science use case connect to what you just showed?"

The demo is a payments API — Python backend, HTML dashboard, SRE incident response. The 40 data scientists at FinTechCo work in Jupyter, pandas, scikit-learn, and TensorFlow. If someone asks how what you showed applies to them, you need a bridge.

The Bridge Statement

"What you just saw was a software engineering workflow — but the capability underneath is the same for your data science team. Claude Code reads your entire project, reasons about it, and takes multi-step actions. For an SWE, that means reading a ticket and shipping a fix. For a data scientist, it means taking a trained model and building the API wrapper, the data pipeline, and the dashboard around it — the work that blocks them from getting their insights into production."

When They Ask "Why Didn't You Demo a Data Science Workflow?"

"I wanted to show the use case with the highest complexity and the clearest compliance constraints — that's your payments infrastructure. The data science workflow is actually simpler to demonstrate because the value proposition is more straightforward: your data scientists spend 30-50% of their time on boilerplate — pipeline setup, API integration, frontend glue code. Claude Code eliminates that."

If They Want a Concrete Example

"One of your fraud detection data scientists has a trained model in a Jupyter notebook. Today, getting that model into a testable API takes a week — FastAPI wrapper, data ingestion pipeline, monitoring dashboard. With Claude Code, they describe what they need and Claude reads the notebook, understands the model's inputs and outputs, and builds the scaffolding. A week of boilerplate becomes an afternoon of review."

Claude Code's Data Science Capabilities

- Reads and writes Jupyter notebooks natively — understands cell structure, interprets outputs including charts
- Converts exploratory notebook code into production-ready scripts and pipelines
- Builds API wrappers around trained models (FastAPI, Flask)

- Handles data pipeline creation (pandas to database to API)
- Turns analysis into shareable dashboards without frontend engineering support

The Head of DT Line

"Your 40 data scientists are probably your highest-leverage people and your most bottlenecked. Claude Code doesn't replace their domain expertise — it removes the 40% of their time that has nothing to do with it. That's the equivalent of adding 16 data scientists without a single new hire."

FEEDBACK AREA 2

Reframe Discovery as "Here's What I've Learned"

Your friend's feedback is right: asking basic discovery questions in a mock interview burns time and signals you haven't done your homework. The interviewers gave you a detailed brief — use it. Frame everything as context you've already gathered, and reference it naturally throughout the presentation.

Old Approach (Cut This)

"Before I show you anything, can I ask a few questions about your code review process?"

New Approach

"I've spent time preparing for this conversation, so let me share what I understand about your situation — and you can tell me where I'm off."

The Opening Script

"Based on what I've learned about FinTechCo: you have 180 engineers across three distinct teams — 120 software engineers working in Python, TypeScript, and Java across digital payments and mobile; 40 data scientists building fraud detection and credit scoring models; and 20 SREs keeping it all running across AWS and GCP. You're looking to roll out a single agentic coding tool company-wide within the next few months, which tells me you want a decision framework, not a pilot that drags on. You're cost-conscious but not cost-driven — you're optimizing for team productivity and shipping velocity, not headcount savings. Does that capture it? And if I can ask one thing: what does success look like for this evaluation in 90 days? I want to make sure my recommendation maps directly to that."

Context You "Already Know" — Weave These In Naturally Throughout

The power move is referencing these details as if you've already discussed them. You haven't literally had a prior call — but the brief gave you the information. Present it like a prepared SA who's done their homework.

Current state and pain points (use during slides):

"From what I understand, your PR cycle has friction points — senior engineer bandwidth creates review bottlenecks, and new engineers take weeks before they're shipping independently. Your data scientists spend a significant chunk of their time on wrangling and pipeline code rather than the actual modeling work they were hired for."

Security posture (use during security slide, looking at CTO):

"I know you don't have any agentic coding tools in place today, which means this would be net-new for your security team. That's actually an advantage — you get to set the governance framework once, correctly, rather than retrofitting after shadow adoption has already started."

Team-specific workflows (use during team breakdown slide):

"Your SREs are managing incident response across AWS and GCP — and I'd bet the hardest part isn't fixing the issue, it's finding the issue in a service they didn't write. Your data scientists are likely working primarily in notebooks, and the gap between a working notebook and a production API is where their projects stall."

Evaluation context (use during competitive slide):

"You're evaluating multiple tools alongside this, and you want a single company-wide standard. That's the right instinct — having three different AI coding tools creates more governance headaches than having one governed one."

Infrastructure (use during demo transition):

"You're on AWS and GCP with internal style guides and CI/CD pipelines. That matters because Claude Code doesn't require any infrastructure changes — it runs in the terminal, works with your existing git workflows, and the CLAUDE.md file I'm about to show you encodes your architectural standards so they're enforced at generation time, not just caught in code review."

Why This Works Better

- Shows preparation and respect for their time
- Demonstrates you understand their business before pitching anything
- Signals enterprise sales maturity — you've done your homework
- Still opens the door for them to correct or add nuance
- The one question you do ask ("What does success look like in 90 days?") is the most valuable one — you'll reference it in your close
- Referencing "what I've learned" throughout the meeting creates the feeling of a tailored recommendation, not a canned pitch

If They Add Context You Didn't Have

"That's really helpful context. Let me make sure I weave that into what I show you."

Write it down visibly. Reference it later. This is better discovery than asking five generic questions.

The Close Callback

Whatever they answer to "What does success look like in 90 days?" — write it down, then reference it verbatim in your wrap-up:

"At the top of the meeting, you said success at 90 days means [their exact words]. Here's how this pilot is designed to get you there..."

This is the single most effective thing you can do to make the presentation feel like a conversation, not a pitch.

FEEDBACK AREA 3

Implementation Complexity and Timelines

This will come up. The CTO wants to know what his security and infrastructure teams need to do. The Head of DT wants to know how fast they can move. The honest answer: the technical complexity is near-zero, and the real timeline is organizational, not technical.

"How Hard Is This to Set Up?"

"A single developer is productive in minutes. Installation is one command — ``curl -fsSL https://claude.ai/install.sh | bash`` — authenticate via browser, and you're working. No IDE plugins to configure, no build system changes, no infrastructure to provision. It works in whatever terminal and workflow your engineers already use."

"What About Enterprise Setup — SSO, Governance?"

"Hours, not weeks. SSO integration is typically 2–4 hours — verify domain ownership via DNS TXT record, upload your IdP metadata to the Admin Console, done. Because it runs through Anthropic's API — or via AWS Bedrock or Google Vertex if you want data to stay in your cloud — there's no self-hosted infrastructure to stand up."

"What Does IT Need to Do?"

"Very little, and that's by design."

Concern	Answer
Network	Outbound HTTPS only. No inbound ports, no firewall changes.
Data residency	Prompt context transits via API, nothing persisted post-session. No training on your code — contractual. Or deploy via AWS Bedrock/Google Vertex to keep all traffic inside your VPC.

Concern	Answer
Provisioning	Per-developer API key managed through your enterprise agreement. Bulk provisioning via the admin console.
SSO	2–4 hours. DNS domain verification + IdP metadata upload. Standard SAML/OIDC.
Governance	One file (CLAUDE.md) and one config (`.claude/settings.json`), both committed to git. Distributed via your existing repo template.
Audit	Every action logged locally. Git history unchanged — Claude's commits look like any other engineer's. Full session audit trails available in enterprise tier.
Rollback	It's a CLI. Uninstall is one command. Nothing stored server-side. Your codebase is unchanged.

What Makes This Low-Complexity vs. Alternatives

"There's no training or fine-tuning required, no data pipelines to build, no model hosting, no IDE lock-in. The CLAUDE.md file — which codifies your team's conventions, security rules, and architectural patterns — is the main configuration artifact. And it's just a markdown file checked into the repo."

For the CTO specifically:

"For a PCI-DSS regulated company, that file becomes the mechanism for enforcing things like 'never hardcode secrets' and 'always use parameterized queries' at the point of code generation — rather than just catching violations in code review. Claude Code doesn't just speed up coding. It shifts compliance enforcement left."

"What's the Timeline to Actually See Value?"

The real timeline is organizational, not technical. A well-run rollout:

Phase	Duration	What Happens
Individual setup	5 minutes	One command, browser auth, working
Enterprise SSO	2–4 hours	Domain verification + IdP metadata
Phase 1: Pilot	Weeks 1–4	3–5 developers per team on real tasks (not toy projects). Build out the CLAUDE.md, document what works, establish prompting patterns. One pilot group from each team: SWE, DS, SRE.
Phase 2: Expand	Weeks 4–8	Share learnings internally, refine governance policies. Pilot devs demo wins to peers — this is where adoption gets momentum.
Phase 3: Broader rollout	Weeks 8–12	Roll out to full 180-person engineering org with established patterns and guardrails. Company-wide go/no-go decision backed by data.

"A single developer is productive in minutes. A pilot team in days. A full org rollout takes 8–12 weeks — but that timeline is dominated by change management and governance, not technical complexity. You said you want a company-wide decision within the next few months. This fits exactly within your window."

"What's the Governance Setup Look Like?"

"Two files. CLAUDE.md is a plain English rules file committed to your repo — your PCI-DSS requirements, authentication patterns, testing standards. Any engineer can read it, your security team writes it. The second is `.claude/settings.json`, which configures governance hooks — automated test execution on every code change, permission modes that control what Claude can do before asking. Both are committed to git and distributed via your repo template. When you roll out to 180 engineers, they inherit the governance on day one. No training sessions."

"Have Other Companies Done This Rollout Successfully?"

"Yes. The pattern we see is that teams who are most skeptical in week one are often the heaviest users by week four — because the governance framework gives them confidence that the tool is operating within their rules. The rollout pattern — start with one team, measure, expand — is specifically designed so you never make a bet bigger than the data supports."

"Does Claude Code Store Our Data? Can It Hold an Entire Repository?"

This is the CTO's question. Get this wrong and the deal is dead. Get it right and you build trust. Think about it in three layers — this maps to how a security-conscious CTO thinks.

The Three-Layer Data Architecture (Know This Cold)

Layer 1: What stays local (never leaves the machine)

"Claude Code stores session history, project configuration, and settings entirely on the developer's machine in `~/claude/`. Session transcripts are saved as JSONL files locally. Project settings live in `~/claude/settings.json` within the repo. The `CLAUDE.md` conventions file is just a markdown file committed to your repo. None of these are stored on Anthropic's servers."

Layer 2: What gets transmitted during a session

"When a developer sends a prompt, the conversation context — including any files Claude Code reads — travels over TLS to Anthropic's API for processing. The response comes back. It's an API call, not a persistent upload. Code is sent for inference, processed, and returned."

Layer 3: What Anthropic retains server-side (and for how long)

"Under enterprise commercial terms: Anthropic does not train models on your code or prompts. Standard retention is 30 days for safety and abuse monitoring, then deleted. For companies that need tighter controls, we offer a Zero Data Retention addendum — conversation data isn't written to disk at all, with abuse checks running only in-pipeline."

"What Happens to Our Code When We Use Claude Code?"

"Claude Code runs in your terminal, on your machine. It reads files locally. When you interact with it, the session prompt is sent to the API over HTTPS. Nothing is persisted on Anthropic's side after the session ends — or within 30 days under standard enterprise terms. There is no database storing your code, no vector store indexing your repo, no backend copy."

"Is Our Code Used for Training?"

"No. And this isn't a settings toggle — it's contractual. Our enterprise DPA explicitly prohibits using customer API inputs for model training. I can get you the specific clause this week."

"Can Claude Code Read Our Entire Repository?"

"Yes, and this is actually one of the key differentiators. Claude Code operates with a 1-million-token context window — roughly 830,000 usable tokens. In practical terms, that's thousands of source files. It can hold both your API layer and the frontend consuming it simultaneously, both the migration and the schema it modifies, both the test suite and the code under test. "For a codebase the size of your payments infrastructure, Claude holds the full picture in context at once. For a very large monorepo — if your entire organization's code is in one repo — it uses intelligent curation: it reads the project structure, focuses on the relevant modules, and pulls in additional files as needed. It doesn't need to load every file to reason effectively about any part of the system."

"What About the Code Context Sent With Each Prompt?"

"Ephemeral. It's processed and discarded per the retention policy — not stored in a searchable index, not used for training. That's a meaningfully different architecture than some competitors."

The AWS Bedrock / Google Vertex Option (Strong for This CTO)

"Since you're already on AWS and GCP, there's another option: deploy Claude Code through AWS Bedrock or Google Vertex AI. This keeps all traffic inside your private VPC — code never traverses the public internet. Your existing AWS compliance controls — CloudTrail logging, IAM policies, VPC isolation — extend naturally to Claude Code usage. For a PCI-DSS regulated environment, this is often the path security teams prefer."

Compliance Certifications

"Anthropic holds SOC 2 Type II, ISO 27001:2022, and ISO/IEC 42001:2023 certifications. For PCI-DSS specifically, the Bedrock deployment path means your existing compliance framework covers Claude Code without a new vendor security assessment from scratch."

"If You Want Belt-and-Suspenders"

"If you want belt-and-suspenders, we can scope the pilot to a non-production repo. But I'd gently push back on that — the value shows up faster on real code, and the security architecture is the same regardless. The other option is starting with the Zero Data Retention addendum so your legal team has the tightest possible terms from day one."

FEEDBACK AREA 5

Other Customers Using Claude Code

Enterprise Adoption Headlines

"70% of Fortune 100 companies use Claude. Over 300,000 business customers. Enterprise accounts generate ~80% of revenue."

Financial Services Customers

Customer	What They're Doing
Intuit	Deploying Claude Code across engineering. Building AI agents with compliance built in.
PwC	Embedding Claude Code where regulatory compliance, auditability, and risk controls are essential.
Accenture	30,000 staff trained. Multi-year partnership. Compliance automation for banks and insurers.
Infosys	Integrating with Topaz platform. Risk detection and compliance reporting in financial services.
Deloitte	Largest enterprise deployment — 470,000 professionals with access.

Developer Adoption Numbers

"JetBrains' January 2026 survey: 18% of developers use Claude Code at work — same as Cursor, with Copilot at 29%. But the growth trajectory matters: Anthropic's revenue went from \$1 billion to \$14 billion in 14 months, driven primarily by enterprise and Claude Code adoption."

Internal Metrics (Anthropic's Own Engineering Team)

"Anthropic's own engineers saw a 67% increase in PRs merged per day since adopting Claude Code. 70-90% of code across their teams is now written with Claude Code assistance."

The ROI Data Point

"Faros.ai, which tracks developer productivity, reports a 4:1 ROI ratio for enterprise teams on Claude Code — measured as cost per incremental pull request. A mid-market company with 300 engineers found 58% of commits were AI-generated and saw an 18% productivity lift."

How to Deploy These in the Meeting

Don't lead with customer names unprompted — it feels like a pitch. Wait for the question, then deploy:

"That's a great question. Let me give you a few reference points..."

If they don't ask, weave one reference into your ROI section naturally:

How to Deploy in the Meeting

Don't lead with customer names unprompted. Wait for the question, then deploy. If they don't ask, weave one reference into your ROI section:

"Anthropic's own engineering team saw a 67% increase in daily PR throughput. I'd expect your pilot to surface a signal in that direction within two weeks."

ROI Framework — How to Measure Claude Code

The Two-Sentence Framework

"We measure two things: how fast your engineers ship code, and how fast they fix what's broken. Everything else rolls up into one of those two."

Shipping Code Quicker — The Metrics

Metric	How to Measure	Typical Baseline	With Claude Code	Signal Timeline
PR cycle time	Open to merge	4-6 hours	2-3 hours	Week 1-2
PRs merged per engineer/day	GitHub analytics	1.2-1.8	2.0-3.0	Week 1-2
Time to first PR (new hires)	Start date to first merge	5-10 days	2-3 days	First new hire
Features shipped per sprint	Sprint velocity	Varies	15-30% increase	Month 1-2

"Anthropic's own engineering team saw a 67% increase in PRs merged per day. I wouldn't promise that for week one, but the directional signal should be visible in your pilot within two weeks."

Debugging Faster — The Metrics

Metric	How to Measure	Typical Baseline	With Claude Code	Signal Timeline
Mean time to diagnose (MTTD)	Alert to root cause identified	30-60 min	10-20 min	First simulated incident
Mean time to resolve (MTTR)	Alert to fix deployed	1-4 hours	30-90 min	First real incident
Bug fix cycle time	Ticket opened to PR merged	1-3 days	2-8 hours	Week 1-2
Security findings per audit	Quarterly audit count	8-15	2-4 (rest caught pre-commit)	Quarter 1

"You just saw a P1 diagnosed and fixed in under 5 minutes — including tests passing and a PR pushed. What does that cycle look like today for your payments team?"

Built-In Measurement Tool

"Claude Code ships with contribution metrics — a built-in analytics dashboard that connects to your GitHub org. Tracks PRs merged with/without Claude Code, lines committed, per-user adoption. Your pilot lead sees the data in real time without building anything."

The Financial Frame (for Head of DT)

"120 engineers recovering 2 hours/week = 240 engineer-hours/week. At \$150/hour loaded cost = \$36,000/week. Annualized = \$1.87 million. The frame isn't headcount reduction — it's shipping velocity. That's the board number."

How to Set Baselines (Do This in Pilot Week 1)

"Baselines are what make or break the ROI story. In week one of the pilot, we measure four things:"

- **PR cycle time** — average of the last 30 PRs per pilot engineer (open to merge)
- **Incident MTTR** — last 10 P1/P2 incidents (alert to resolved)
- **Onboarding time** — start date to first meaningful commit (ask the last 3 new hires)
- **Security findings** — count from the last internal security review

"At the end of the pilot, you compare against these baselines with real data. Not a vendor case study — your data, your engineers, your codebase."

If They Ask "What Does Bad Look Like?"

"If the pilot data doesn't show improvement, I will tell you. My job is to help you make a good decision, not a fast one. The pilots that don't succeed usually fail on one of two things: governance wasn't set up before engineers started, so they had a bad first experience — or the use cases were too narrow to show signal. That's why I recommend specific, high-friction workflows for the pilot, not 'play around and tell us what you think.'"

BONUS

Handling Verbose Output During the Demo

Prevention: /fast mode is on (pre-demo checklist). Prompts are tuned. If testing reveals verbosity, add "Be concise" to the prompt.

If Output Is Long:

"Claude gave a thorough analysis — let me pull out the key findings."

Scroll to the summary. Never read entire output. Narrate value, don't read a screen.

If They Comment:

"Engineers tune this to preference — concise mode for quick fixes, full detail for security audits. For the audit we just ran, I actually want the detail."

BONUS

"You're Not Using Our Actual Codebase"

Proactive Framing (Before the Demo):

"The codebase I'm showing you is a payments API I built for this meeting — PCI-DSS constraints, Python backend, multi-service architecture, real security issues. I designed this to mirror the complexity your payments engineers deal with daily."

If They Push:

"That's exactly what the pilot is for. Week one, your engineers run Claude Code against your real codebase. The demo shows capability; the pilot proves it in your environment."

Stronger Version (Skeptical CTO):

"This demo is harder than a polished production codebase — I intentionally loaded it with vulnerabilities that would never pass your code review. The point is the workflow: describe the problem, Claude reads the codebase, reasons, implements, tests, and proves it. That workflow is codebase-agnostic."

QUICK REFERENCE

Six Questions, One-Line Anchors

Question They'll Ask	Your Anchor
"How does data science apply?"	Same capability, different workflow. DS boilerplate becomes model time.
"Why ask discovery questions you should know?"	Don't. Lead with "Here's what I've learned about you."
"How complex is implementation?"	One command to install. Two files for governance. Day 1 to value.
"Do you store our code?"	Nothing persisted post-session. Contractual. SOC 2 Type II.
"Who else is using this?"	70% of Fortune 100. Intuit, PwC, Accenture in financial services.
"How do we measure ROI?"	Two things: shipping speed and debugging speed. Built-in metrics dashboard.

Last updated: April 2026 | Supplements DEMO_PREP_v4.md + DEMO_CHEATSHEET.md + INTERVIEW_PREP.md